

src/equilibration.cpp — Function-by-Function Explanation

A-Phi Solver project — code reference

September 2026

Contents

Overview	1
<code>compute_symmetric_equilibration</code>	1
<code>apply_symmetric_equilibration</code>	2
<code>scale_rhs</code>	2
<code>recover_equilibrated_solution</code>	2

Overview

This file is purely numerical linear-algebra machinery, unrelated to the electromagnetic formulation itself: it rescales a dense complex matrix before factorization/solving, purely to improve its numerical conditioning. The header (`equilibration.hpp`) is explicit that this is a distinct concern from `conditioning.hpp`, which fixes the *physics/frequency-domain* scaling of the A-Phi system (things like the $j\omega$ row scaling and the $\Phi = j\omega\Phi$ substitution). This file instead fixes the *numerical scale* of whatever matrix it is handed — the two can be applied together, in either order.

Why symmetric scaling specifically: the A-Phi system's matrix is complex *symmetric* ($A = A^T$, not Hermitian), and that structure matters downstream — it is what makes symmetric-indefinite factorizations and COCG/COCR-type iterative solvers usable at all (planned for Phase 05). A generic row-and-column (Ruiz) equilibration would generally use two different scaling vectors and destroy that symmetry; this file instead uses a *single* diagonal vector d applied on both sides at once, $\text{diag}(d) * A * \text{diag}(d)$, which preserves symmetry exactly because $\text{diag}(d)$ equals its own transpose.

There are four functions in the file, covered below in the order they appear, which is also the order they are meant to be used in: compute a scaling vector, apply it to the matrix, apply the matching scaling to the right-hand side, solve, then undo the scaling on the solution.

`compute_symmetric_equilibration`

```
std::vector<double> compute_symmetric_equilibration(const ComplexMatrix& A, int iterations)
```

Computes a real, positive diagonal scaling vector d (one entry per row/ column of the square matrix A) via a symmetric Ruiz-style iterative scheme, without modifying A itself.

Validation: it first checks `A.cols() == A.rows()` (the scaling only makes sense for a square matrix) and throws `std::invalid_argument` otherwise.

The iteration: d starts as all-ones, and a working copy of A is progressively rescaled across iterations passes (default 10, per the header comment — enough for the scaling to converge in practice, with more rarely helping). Each pass:

1. Finds, for every row i of $working$, the largest-magnitude entry in that row ($row_max[i]$) — since the matrix is kept symmetric throughout, this is simultaneously each row's *and* each column's largest entry.
2. Sets that row's scaling factor $[i] = 1 / \sqrt{row_max[i]}$ — the square root is what makes the *combined* effect of scaling both the row and the column by this same factor bring the row's largest entry down toward 1, rather than overshooting. A row whose max is already 0 (a zero row) is left with $factor[i] = 1$ instead of dividing by zero — the comment notes this shouldn't happen for a well-posed system, but is handled safely rather than crashing if it does.
3. Multiplies the running total $d[i] *= factor[i]$ (accumulating this pass's correction into the overall scaling built up so far).
4. Applies the rescaling to $working$ itself, $working(i, j) *= factor[i] * factor[j]$, so the next pass's row_max reflects the already-partially-equilibrated matrix.

After all passes, d is returned — the vector that, applied via `apply_symmetric_equilibration` below, is intended to leave every row/column of the rescaled matrix with a largest-magnitude entry close to 1.

apply_symmetric_equilibration

`ComplexMatrix apply_symmetric_equilibration(const ComplexMatrix& A, const std::vector<double>& d)`

Returns a new matrix equal to $\text{diag}(d) * A * \text{diag}(d)$ — that is, entry (i, j) of the result is $A(i, j) * d[i] * d[j]$.

Validation: throws `std::invalid_argument` if A isn't square or if d 's length doesn't match A 's dimension.

This is the actual rescaling step applied once, using the d that `compute_symmetric_equilibration` computed — as opposed to that function's own internal `working` copy, which is just an intermediate used to *derive* d and is discarded afterward. The real system that eventually gets factored and solved is the one this function returns.

scale_rhs

`ComplexMatrix scale_rhs(const ComplexMatrix& b, const std::vector<double>& d)`

Returns $\text{diag}(d) * b$, i.e. row i of b scaled by $d[i]$ (with b possibly having multiple columns — e.g. multiple right-hand sides solved at once — since the scaling is applied per row regardless of column count).

Validation: throws `std::invalid_argument` if d 's length doesn't match b 's row count.

This is the second half of turning the original problem $A x = b$ into an equilibrated one: having replaced A with $\text{diag}(d) A \text{diag}(d)$ via `apply_symmetric_equilibration`, the right-hand side needs the matching $\text{diag}(d)$ scaling too, so that solving $(\text{diag}(d) A \text{diag}(d)) y = \text{diag}(d) b$ for y is consistent with the original system under the substitution $x = \text{diag}(d) y$ (see `recover_equilibrated_solution` below).

recover_equilibrated_solution

`ComplexMatrix recover_equilibrated_solution(const ComplexMatrix& y, const std::vector<double>& d)`

Returns $\text{diag}(d) * y$ — after the equilibrated system $(\text{diag}(d) A \text{diag}(d)) y = \text{diag}(d) b$ has been solved for y , this recovers the actual solution x to the original, unscaled problem $A x = b$, via the substitution $x = \text{diag}(d) y$.

The implementation is a one-line call straight to `scale_rhs(y, d)` — it is literally the same elementwise `diag(d)` multiply. It is kept as its own named function rather than reusing `scale_rhs` directly at call sites purely for readability: at the point this is called, the operation means “undo the substitution and recover the physical solution”, which is a different intent from “scale a right-hand side before solving”, even though the arithmetic performed is identical.